ENTERPRISE DATA RECONCILIATION

Financial Audit and Revenue Analysis Using Python, SQL and Data Visualization

Project type	Data Analytics / Financial Reconciliation
Reporting period	1 January – 31 December 2023
Source systems	SQLite database, server logs, daily EUR/USD exchange rates
Technology stack	Python, Pandas, SQLite, Regular Expressions, Matplotlib
Deliverable	Reconciled transaction dataset and CFO dashboard

Prepared from the supplied project database, server logs, exchange-rate CSV, notebook, and project brief.

1. Executive Summary

This project implements a repeatable data reconciliation pipeline across three source systems: a SQLite customer database with historical account records, unstructured server transaction logs, and daily EUR-to-USD exchange rates. It selects the latest account status per user, excludes error events, extracts transaction fields with regular expressions, fills missing exchange rates, and calculates USD revenue for transactions linked to users whose latest status is Active.

Metric	Observed result
Raw server log records	100,000
Error records excluded	5,098
Valid transactions parsed	94,902
Historical user records	47,464
Distinct users in database	25,000
Latest active users	14,183
Missing exchange rates filled	104
Transactions reconciled to active users	53,899
Active customers with reconciled transactions	13,888
Total reconciled revenue	$148,274,622.91
Pipeline processing time (observed run)	1.739 seconds

Note: processing time is a measured result from one local run and may vary with hardware, Python version, and disk speed.

2. Problem Statement and Objectives

The source data is distributed across systems and is not immediately analysis-ready. Server logs store transaction attributes inside text strings; user records contain multiple historical versions; and daily exchange-rate values are missing on some dates. A naive join or row-by-row parsing approach could lose records or be inefficient.

Project objectives

• Extract each user's latest account status using a SQL window function.
• Exclude failed log events and parse valid transaction fields from text.
• Reconcile transaction dates with daily currency rates while preserving dates with missing rates.
• Convert EUR amounts to USD using the date-matched exchange rate.
• Restrict the final financial dataset to users whose latest status is Active.
• Produce monthly revenue summaries and a top-five customer analysis.

3. Input Data and Tools

Input	Contents and role
enterprise_database.db	SQLite users table; historical user status and update timestamps.
server_logs.txt	100,000 raw log lines containing date, user ID, product ID, amount, and log level.
daily_exchange_rates.csv	365 daily records for 2023 with Date and EUR_to_USD fields; some rates are missing.
Student_work_file.ipynb	Starter notebook organized into SQL extraction, log parsing, reconciliation, and visualization phases.
statement.pdf	Project brief describing the scale, unstructured-data, time-series, and historical-version challenges.

4. Project Architecture and Workflow

Source files → SQL and log extraction → data quality checks → date-based exchange-rate merge → active-user reconciliation → USD revenue calculation → aggregations and dashboard.

Phase 1 — SQL extraction

The query ranks each historical row within its user group, newest update first. It then selects rank 1 and keeps the user only if that latest row has status Active. This ordering avoids incorrectly retaining a customer who was active earlier but later became inactive or suspended.

```
WITH ranked_users AS (
    SELECT user_id, name, status, updated_at,
           ROW_NUMBER() OVER (
               PARTITION BY user_id
               ORDER BY updated_at DESC
           ) AS rn
    FROM users
)
SELECT user_id, name, status, updated_at
FROM ranked_users
WHERE rn = 1 AND status = 'Active';
```

Phase 2 — vectorized server-log parsing

The pipeline reads the log file as text, filters lines containing ERROR, and uses a named-group regular expression with Pandas string extraction to retrieve Date, User_ID, Product_ID, and EUR_Value. Numeric and date fields are converted to their appropriate types. Vectorized string operations avoid slow Python loops over individual rows.

```
pattern = (
    r'\[(?P<Date>\d{4}-\d{2}-\d{2})\s+\d{2}:\d{2}:\d{2}\]'
    r'.*?"u":"(?P<User_ID>U-\d+)"'
    r'.*?"item_code":"(?P<Product_ID>PRD-\d+)"'
    r'.*?"eur_val":(?P<EUR_Value>\d+(?:\.\d+)?)'
)
extracted_data = info_logs.str.extract(pattern)
```

5. Financial Reconciliation

Exchange-rate preparation

Exchange rates are sorted by date and missing values are filled with the previous available value using forward-fill. This handles weekends and other missing dates according to the supplied project requirement. A left join is used at the exchange-rate stage so a valid transaction is not silently discarded simply because the join is being performed.

```
rates = pd.read_csv(RATES, parse_dates=["Date"]).sort_values("Date")
rates["EUR_to_USD"] = rates["EUR_to_USD"].ffill()
with_rates = extracted_data.merge(
    rates, on="Date", how="left", validate="many_to_one"
)
```

Active-user reconciliation and conversion

The transaction records are inner-joined to the selected latest-active user records. The conversion is calculated as EUR_Value multiplied by EUR_to_USD. The pipeline derives a Month field from the transaction date for monthly aggregation.

```
df_final = with_rates.merge(
    df_active_users[["user_id", "name", "status", "updated_at"]],
    left_on="User_ID", right_on="user_id",
    how="inner", validate="many_to_one"
)
df_final["USD_Revenue"] = df_final["EUR_Value"] * df_final["EUR_to_USD"]
df_final["Month"] = df_final["Date"].dt.to_period("M").astype(str)
```

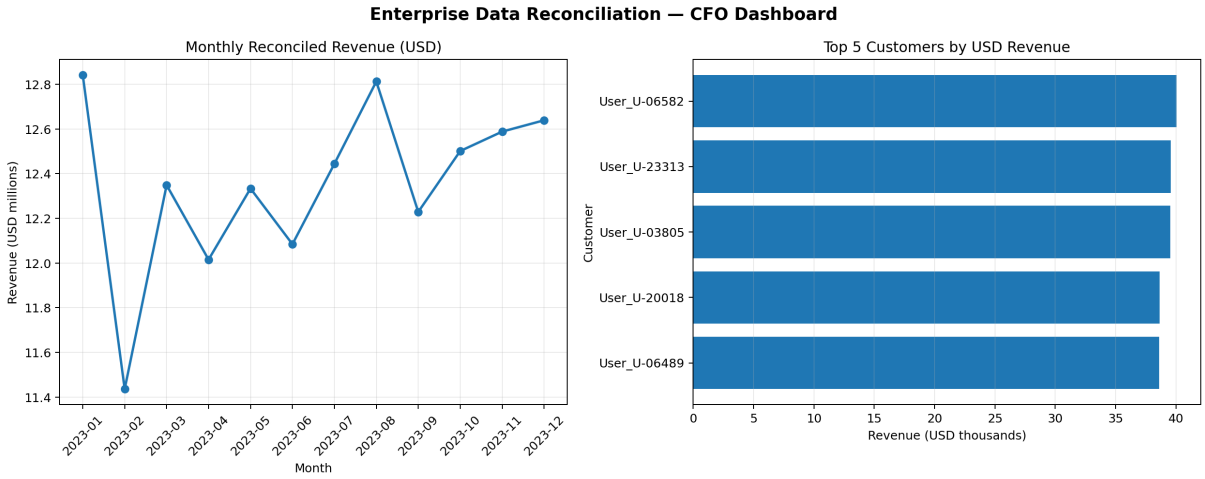
Data-quality checks

The pipeline verifies that all transaction dates have an exchange rate after filling, uses merge validation to check expected many-to-one relationships, and records the counts of raw logs, errors, parsed transactions, active users, and reconciled transactions. The final output and aggregation CSVs are written to the output folder.

6. Results and Visual Analysis

The run processed 100,000 log records. It excluded 5,098 ERROR records and parsed 94,902 valid transactions from the remaining logs. The database contained 47,464 historical user rows for 25,000 distinct users. The latest-status query selected 14,183 active users.

The exchange-rate file had 104 missing values filled. After matching transactions to users whose latest status was Active, the final dataset contained 53,899 transactions for 13,888 distinct customers. Total reconciled revenue was $148,274,622.91.



Monthly revenue (USD)

Month	Revenue (USD)
2023-01	$12,841,761.38
2023-02	$11,436,130.90
2023-03	$12,348,931.84
2023-04	$12,015,376.19
2023-05	$12,333,644.45
2023-06	$12,084,355.82
2023-07	$12,444,163.58
2023-08	$12,812,730.66
2023-09	$12,228,952.05
2023-10	$12,501,330.36
2023-11	$12,588,223.09
2023-12	$12,639,022.60

7. Top Five Customers by Reconciled USD Revenue

The following customers have the highest summed USD revenue in the reconciled dataset. Customer IDs and names are shown as present in the supplied data.

User ID	Customer name	Revenue (USD)
U-06582	User_U-06582	$40,050.99
U-23313	User_U-23313	$39,555.81
U-03805	User_U-03805	$39,529.95
U-20018	User_U-20018	$38,649.97
U-06489	User_U-06489	$38,629.14

Key findings

• The log-processing approach handled 100,000 rows in an observed 1.739 seconds on this run, using vectorized operations rather than row-by-row iteration.
• 5,098 error events were excluded from valid transaction extraction.
• Historical user-version handling reduced 47,464 rows to one latest record per user, then retained only Active users.
• Exchange-rate forward-fill handled 104 missing daily values, allowing date-based reconciliation.
• The final active-user transaction dataset totals $148,274,622.91 in calculated USD revenue.

8. How to Run the Project

1. Install Python 3 and the required packages: pandas and matplotlib (SQLite support is included with Python).
2. Keep the notebook, build_project.py, database, server logs, and exchange-rate CSV in the same project folder.
3. Run `python build_project.py` from that folder to regenerate the outputs.
4. Open `Enterprise_Data_Reconciliation.ipynb` in Jupyter Notebook or VS Code to review the phases interactively.
5. Check the `output/` folder for cleaned data, final reconciled transactions, monthly revenue, top-five customers, metrics, and dashboard image.

Main output files

• `output/active_users.csv` — latest active user records.
• `output/clean_transactions.csv` — parsed non-error transactions.
• `output/final_reconciled_transactions.csv` — transactions joined to exchange rates and active users, with USD revenue.
• `output/monthly_revenue.csv` — monthly revenue summary.
• `output/top_5_customers.csv` — five highest-revenue customers.
• `output/cfo_dashboard.png` — two-panel CFO dashboard.
• `output/project_metrics.json` — machine-readable processing metrics.

9. Assumptions and Limitations

• The report follows the supplied project brief: ERROR log entries are excluded from transaction parsing.
• Forward-fill assumes the most recent available exchange rate remains applicable until a new rate is present; this is the project's specified reconciliation rule, not a claim about every market or accounting policy.
• Revenue is calculated as transaction EUR value multiplied by the supplied EUR-to-USD rate. Fees, refunds, tax, chargebacks, and product costs are not modeled unless represented in the supplied inputs.
• The reconciliation includes transactions only for users whose latest database status is Active. Transactions for users with other latest statuses are intentionally not included in the final active-user dataset.
• Processing time is machine-dependent and should be re-measured in the submission environment.

10. Conclusion and Future Scope

The project demonstrates a practical data analytics workflow that combines SQL window functions, regular-expression extraction, vectorized data processing, time-series handling, relational joins, aggregation, and visualization. It transforms three heterogeneous source systems into a reconciled transaction dataset and summary dashboard. Future improvements could include automated anomaly detection, validation rules for duplicate transactions, a scheduled ETL job, interactive dashboard filters, and audit trails for records excluded during reconciliation.

References

1. Supplied project brief: "Pedagogical Analysis: Why the Enterprise Data Reconciliation Project is Difficult."
2. Supplied starter notebook: Student_work_file.ipynb.
3. Python standard library documentation for sqlite3 and re; Pandas documentation for string extraction, joins, and forward-fill; Matplotlib documentation for chart creation.